Concurrency and Distributed Systems

Week 6 – Part 1: Fork/Join Model

Dr. Mohamad Aoude

Faculty of Engineering

May 13, 2026

Part 1 Roadmap

1. From Execution to Decomposition
2. RecursiveTask
3. Work Stealing
4. Thresholds
5. Lab Preparation

Why This Part Matters

Core question

When is a computation worth splitting across cores?

Week 5 controlled task submission with bounded queues and pools. Week 6 is different: we split one CPU-bound computation into independent subtasks.

Concurrency vs Parallelism

Concurrency	**Parallelism**
Manage many tasks in progress	Execute multiple parts at the same time
Often about waiting and coordination	Often about CPU cores and decomposition
Week 5: bounded executors	Week 6: Fork/Join

Sequential Baseline

```
long total = 0;
for (int value : values) {
    total += value;
}
return total;
```

Rule

Always build and test the sequential version first. It is the correctness reference and the performance baseline.

RecursiveTask Shape

```
protected Long compute() {
    if (smallEnough()) {
        return computeSequentially();
    }

    Task left = new Task(leftRange);
    Task right = new Task(rightRange);
    left.fork();
    long rightValue = right.compute();
    long leftValue = left.join();
    return leftValue + rightValue;
}
```

Why Fork One Side and Compute the Other?

- Forking both sides can create unnecessary queued work.
- Computing one side keeps the current worker productive.
- Joining waits for the forked side only when its result is needed.

Pattern

Fork one subtask, compute the other subtask directly, then join.

Work Stealing

- Each worker has a deque of tasks.
- A busy worker pushes and pops local subtasks cheaply.
- An idle worker can steal work from another worker.
- This helps balance uneven recursive splits.

Boundary

Work stealing helps CPU-bound independent work. It does not make blocking I/O safe.

Threshold Selection

- Too small: many tiny tasks, scheduling overhead dominates.
- Too large: not enough parallelism, cores stay idle.
- Good threshold: enough work per task to repay split and join costs.

Exercise link

`Ex1_ForkJoinSum` implements the split. `Ex2_BreakEvenReport` measures when it pays off.

Lab Demos

- `Demo01_SequentialArraySum`: correctness and baseline.
- `Demo02_ForkJoinArraySum`: `RecursiveTask` with threshold.
- `Demo03_ForkJoinPoolContext`: common-pool parallelism and processor count.

Exit Check

1. What is the base case in a Fork/Join task?
2. Why should the base case compute sequentially?
3. Why is blocking I/O not a good fit for Fork/Join?